

Application: are unlearned capabilities still in the weights?

The elicit-vs-teach metrics on WMDP-unlearned Zephyr-7B

Question. The main results show that the metrics separate a capability that is already in the weights (fine-tuning *elicits* it) from one that is absent (fine-tuning *teaches* it). Unlearning claims to move a model from the first case to the second. If it did, relearning would be teaching; if the capability is only hidden, relearning is elicitation, and the metrics should say so without any training at all.

Setup in six lines. The original model is Zephyr-7B-beta (32 layers, 32 heads per layer). Four public unlearned versions of it are tested: RMU, ELM, NPO, SimNPO (pinned checkpoints). The capability is WMDP-bio, 4-option multiple choice, scored as the argmax over the four answer letters (chance 0.25). The items are split at random into halves A and B. *Relearning* fine-tunes on A as text facts (question and correct answer, no options, so no letter can be learned; LoRA $r=64$, learning rate 3×10^{-5} , the checkpoint with the lowest validation loss is kept) and is read on the held-out half B. The *fine-tuning null* is the same recipe on MMLU facts, which cannot teach bio answers. There is no never-learned model: every metric is read against the model’s *own* null and against the original.

Reading a cell. r is the position between the model’s own null (0, the teach end) and the original (1, the elicit end); the interval is 90% where the statistic has a standard error. **E:** $r \geq 0.5$. **R:** above its own null, $r < 0.5$. **A:** at its own null. The metric numbers are those of the main results.

Headline: does the capability come back, and what brings it back?

	RMU	ELM	NPO	SimNPO
accuracy on B, unlearned model	0.31	0.35	0.26	0.42
after the fine-tuning null (MMLU facts)	0.35	0.55	0.62	0.62
after relearning on A (bio facts)	0.68	0.67	0.64	0.66
M17u any fine-tune unlocks B, r [90%]	A 0.11 [−0.02, 0.23]	E 0.64 [0.50, 0.78]	E 0.89 [0.78, 1.00]	E 0.80 [0.62, 0.99]
M17 recovery beyond the null, r [90%]	E 1.04 [0.88, 1.19]	E 0.98 [0.57, 1.40]	— (no room)	— (no room)
M18 relearning cost over the original, net of the null (nats/token)	+0.15 $\pm$ 0.19	+0.03 $\pm$ 0.14	+4.6 $\pm$ 9.0	−0.31 $\pm$ 0.53
MMLU accuracy (original 0.58)	0.58	0.59	0.47	0.48

Table 1: The original model scores 0.67 on B. M17u reads the null child against the unlearned model; M17 reads the relearned child against the null child. When the null already reaches the original (NPO, SimNPO) M17 has no room to measure and its interval spans the whole range; M17u is the informative one there. M18 is the online code length of relearning (each batch scored before the model trains on it) minus the original’s, net of the same difference on the null; $\pm$ is 90%.

- **Every method leaves the capability in the weights.** Relearning half of the items restores the other half to the original’s accuracy (0.64–0.68 vs 0.67). The facts it trains on cannot teach the held-out answers.
- **They differ in what unlocks it.** For NPO and SimNPO an unrelated fine-tune on MMLU facts alone brings back most of it ($r = 0.89, 0.80$). For RMU it does almost nothing ($r = 0.11$,

interval includes 0); only bio-specific training opens it, and then fully ($r = 1.04$). ELM is in between.

- **Relearning costs what the original’s own relearning costs.** RMU $+0.15 \pm 0.19$, ELM $+0.03 \pm 0.14$ nats/token: both intervals include 0, the elicit end. NPO’s first steps repair a broken output (validation loss starts at 240 nats), so its interval is too wide to read.

Every metric, before and after relearning

metric (main results)		RMU	ELM	NPO	SimNPO
<i>Parent-only (★): the unlearned model itself, no training</i>					
M12	hidden preference, bio	R 0.25	R 0.12	R 0.13	E 1.06
M12	hidden preference, cyber	R 0.09	R 0.04	E 0.65	E 0.93
M16'	answer probe, bio	A	A	E 0.59	E 0.67
M16'	answer probe, cyber	A	A	E 0.73	E 0.61
M9	answer depth (lens)	R 0.24	R 0.08	R 0.04	R 0.09
M11	repeatable task circuit	A (n)	A (n)	A (n)	E 0.68
M14	answer carried by the state	A	A	A	E 0.57
M15	option-reading heads	E 0.70	A	A	E 0.96
M1★	original's heads necessary in U	E 1.06 [†]	n	n	E 0.89
M10★	original's head states patched into U	R 0.33	E 0.92	A	R 0.28
M2★	wiring, U vs original	n	n	n	R 0.41
M13	accelerating descent	pending (block Hessian, blocks 20 and 28)			
<i>After relearning on A, read on B: the relearned child vs the original</i>					
M1	circuit overlap (heads, J@32)	E 1.22	E 1.07	E 1.14	E 1.07
M1	original's heads necessary in the child	E 1.02	E 0.99	E 0.99	E 0.96
M2	wiring (edges)	E 1.18	E 1.24	E 1.31	E 1.01
M3	head roles	R 0.12	A	R 0.17	R 0.14
M4	circuit present from the first snapshot	R 0.47	E 1.00	E 0.60	R 0.49
M10	child's head states patched into U	A	E 0.88	A	A
<i>Descriptive, no own null</i>					
M7★	orig → U: one shared push (PC1, ≥0.67 = shared)	0.40	0.65	0.84	0.60
M7	U → child: per-item change (<0.5)	0.49	0.57	0.69	0.66
M5	gradient fades (first 1% / last 10%)	14.5	6.1	134	13.7
M6	weight write vs the original's relearning	1.01×	1.00×	1.27×	1.25×

Table 2: Position r for each metric; n = the model has no task behaviour for the metric to read (mean logit difference < 1 nat), which for M11 is itself the reading (A). Reference values for the original model: M12 3.33 / 3.57 nats, M16′ 0.60 / 0.46, M9 4.88 nats, M11 0.64 (split-half), M14 0.84, M15 0.73, M1★ 0.93; the relearned children are compared with the original’s own relearned child (M1 ceiling 0.60, M2 ceiling 0.25–0.29). M16′ adapts M16: the probe reads the correct option, not the operands. M8 (latent reach) has no WMDP analogue and is not run. [†] M1★ for RMU is a fraction of a small signal (logit difference $+0.75$ vs $+4.87$ in the original). Intervals for every row are in `verdict.wmdp.md`; binomial rows are in Table 1.

- **After relearning, all four look elicited.** The child’s circuit, its wiring and the original’s heads’ necessity all match the original’s own relearned child ($r \approx 1$ on M1, M2), exactly as the elicited child did in the main results. Head roles (M3) do not carry over for any method; the circuit is the same, the individual reader heads are not.

- **Before any training the methods separate.**
 - *SimNPO*: nine of twelve parent-only metrics read like the original. The circuit, the state and the heads are intact; only the output is pushed away from the answer.
 - *NPO*: the answer is readable in the state (M16') while the output is broken, and the change from the original is one shared push (M7★ 0.84): suppression at the read-out.
 - *ELM*: no probe or circuit reading, but patching the original's head states back in turns the capability on ($r = 0.92$).
 - *RMU*: the answer is not readable anywhere before training (probe, lens, circuit at or near null). It is the one method that hides the capability from the parent-only metrics, and still it comes back in full on relearning at the original's cost.

Caveats

- There is no teach anchor: without a model that never learned WMDP-bio, “costs what the original's relearning costs” and “ $r \approx 1$ ” are statements relative to the original, not to a measured teaching cost.
- Parent-only metrics are read on all bio items, relearning metrics on half B; the halves are random but the first 512 items in source order score lower (original 0.51) than B (0.67), so the two blocks are not compared with each other.
- M5 and M6 read the same for every method and do not discriminate here; LoRA fixes the size of the weight write.
- One seed, one relearning recipe; the relearning curve is noisy at step level (held-out accuracy moves by ± 0.1 between evaluations), which is why the minimum-validation-loss checkpoint is used rather than the last one.